

DEEPPFAKE CLASSIFICATION PROJECT REPORT

Perli Davide-Andrea

Contents

- CNN Model Description2
 - 1. Reasoning:.....2
 - 2. Data Reading and Processing:.....2
 - 3. CNN Design:2
 - 4. Training3
 - 5. Testing.....5
 - 6. Validation5
 - 7. Data Augmentation.....5
 - 8. Confusion Matrix.....8
- K-NN Model Description.....9
 - 1. Reasoning.....9
 - 2. Data Reading and Processing9
 - 3. Training + Testing9
 - 4. Validation9

CNN Model Description

1. Reasoning:

I chose a convolutional neural network as the main model since it's the best for image based training, even though it's much slower than a K-NN (my second model), mainly because I implemented 10 layers.

2. Data Reading and Processing:

After reading the training data along with the corresponding labels, I chose as an approach to normalize the images, by making the pixels to take float values from 0 to 1 instead of from 0 to 255 as a normal image would. I convert the images from one color space to another (from BGR to HSV). The first one I used for a lot of time was BGR to RGB since it's a classic and basic one, but after testing on my second model where I saw an improvement of 40% by using BGR to HSV I decided to stick with that since it keeps since it's better at color separation. My first approach was to split the image into 3 channels r, g, b and normalize each one (with norm MinMax), but after testing by normalizing the image as a whole and seeing an improvement of 2% I decided to stick with the later one, also due to the fact that after searching the matter I discover that it keeps the images more stable. Some first attempts also included resizing the images to 32x32, but the validation accuracy was quite low so I manually checked what dimension where the images with `image.shape` and I've discovered that all of them where 100x100 with 3 channels so I decided to keep the original resolution since I saw an improvement of at least 10% - 12%, even though it's more computationally expensive. Still I chose to resize each image to 100x100 in order to make sure the model would work just fine without any problems even if I got an image with a different size as an input.

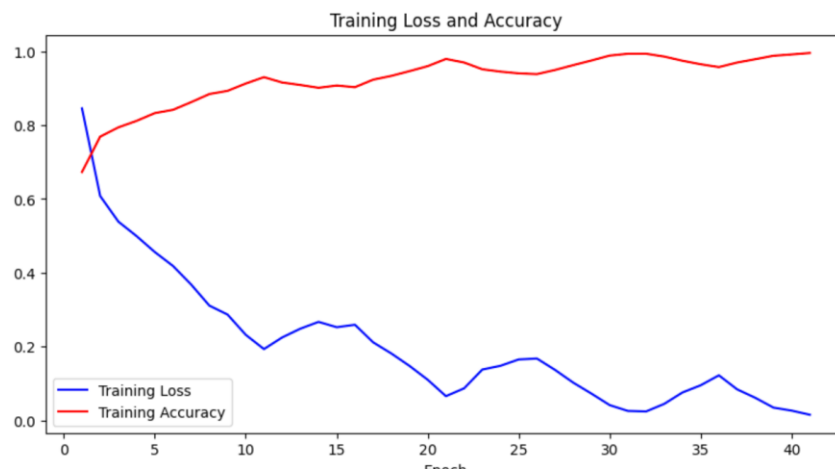
3. CNN Design:

I used PyTorch for this model rather than Keras since it has higher speeds, which helps with performance and I don't have a powerful machine, also it's better supported since it has a bigger community. At the begging I had 2 convolution layers (first with 3 input channels, 6 output and 5x5 kernel size, second with 6 input channels (next layers take the previous one output channels as input channels), 16 output channels and 5x5 kernel size) with a pooling layer (after the first convolution layer with 2x2 kernel size and stride = 2 since I wanted to cut in half the width and height) and 2 fully connected layers (first: input 16 x 5 x 5 flattened after the convolution layers and pool layers, output 120 neurons, second: 120 neurons and 84 neurons as output) and for the output layer: 84 neurons, 10 neurons which was like those studied during the labs using MNIST class. At the forward function after each convolutional layer I added ReLU activation function and then max pooling. After that I needed to flatten in order to feed an array of pixels instead of pixel values instead of the 2D image into the fully connected layers where I apply a linear transformation ($y = W * x + \text{bias}$) with batch size = 400, in the second one is 84 and in the third one is 10. I added ReLU after each convolution layer and fully connected layer to help gradient flow better. Later I decided to change the structure by 10 layers (in 4 blocks) all with kernel size = 3 since it's better for feature extraction (from the 3x3 matrix features it extracts a feature,

5x5 lost a lot of important features) padding = 1 for default and bias = False since I normalize after each layer and it's not necessary. The first and second block have 2 layers while the third and fourth block have 3 layers. The last layer stops with 256 out channels. I tried to add a 5-th block with another 3 layers with 512 out channels each and the first layer with 256 channels in, but the accuracy dropped since I lacked a residual block to stop the vanishing gradient problem. I didn't try that approach since I already had a high validation accuracy. In this updated version I use max pooling after each block to cut the dimensions in half and after all 4 block a global average pooling = nn.AdaptiveAvgPool2D(1) (since adaptive pooling makes it input-size flexible and I was still playing with image resizing and so I didn't have to calculate for each time the values) necessary in order to replace fully connected layers which I previously had, then flatten it and applied dropout (0.5 was optimal after testing other values) since it's a regularization technique and then apply a linear transformation and then output the class scores.

4. Training

After normalization/resizing I convert the image from a numpy array to a PyTorch tensor, with permute (2, 0, 1) change the order of dimensions (height, width, channels to channels, height, width) since in the CNN the convolutional layers expect the image format to be like that add the image_tensor and label to a tuple. Load them in a tensor dataset object in order to feed them to the train loader with 32 samples at a time for each batch, shuffle to improve generalization and 8 workers (that's the number of subprocesses that I found to be the sweet spot). Choose the device where to run, in my case it runs on CUDA since I have an RTX 3050. I chose to run for a large number of epochs (200) and plot the accuracy and loss for each epoch in order to find the optimum number of epochs by looking at the plot and choosing the epoch where the loss begins to be a straight line since I don't want to have overfit. Another approach taken was to stop when the training accuracy reached a certain level. Here is an example of that:



Also here can be seen some of the batches with each loss/accuracy per batch and per epoch:

```

Batch 300, Loss: 0.0619
Epoch 37 Loss: 0.0841, Accuracy: 0.9697
=== Epoch 38 ===
Batch 0, Loss: 0.0543
Batch 100, Loss: 0.0206
Batch 200, Loss: 0.1030
Batch 300, Loss: 0.0459
Epoch 38 Loss: 0.0610, Accuracy: 0.9787
=== Epoch 39 ===
Batch 0, Loss: 0.0596
Batch 100, Loss: 0.0130
Batch 200, Loss: 0.0060
Batch 300, Loss: 0.0224
Epoch 39 Loss: 0.0345, Accuracy: 0.9881
=== Epoch 40 ===
Batch 0, Loss: 0.0062
Batch 100, Loss: 0.0017
Batch 200, Loss: 0.0115
Batch 300, Loss: 0.0556
Epoch 40 Loss: 0.0263, Accuracy: 0.9916
=== Epoch 41 ===
Batch 0, Loss: 0.0101
Batch 100, Loss: 0.0050
Batch 200, Loss: 0.0270
Batch 300, Loss: 0.0056
Epoch 41 Loss: 0.0151, Accuracy: 0.9956
Stopping early at epoch 41 as accuracy is 0.9956
Finished Training

```

As a loss function I used the Cross Entropy Loss. For each epoch I initialize the sum of losses over all batches for the current epoch, the number of correctly predicted labels and the total number of samples up to that point. Then I loop over the small batches determined here: `train_loader = DataLoader(dataset, batch_size=32, shuffle=True, num_workers=8)`. I tried increasing the batch size over at 64, but I saw a small decrease in accuracy at validation. The data is moved to GPU/CPU. I then use the optimizer with `set_to_none` as a parameter in order to clear the gradients, then I forward pass to the model the inputs, compute the loss, backpropagation that calculated gradients, updating the weights and adjust the learning rate. All of this can be seen here:

```

inputs = inputs.to(device)
labels = labels.to(device)
optimizer.zero_grad(set_to_none=True)
outputs = model(inputs)
loss = loss_function(outputs, labels)
loss.backward()
optimizer.step()
scheduler.step()

```

Also here are the optimizer function and the scheduler:

```

optimizer = torch.optim.SGD(model.parameters(), lr=0.001, momentum=0.82,
weight_decay=1e-4, nesterov=True)
scheduler = torch.optim.lr_scheduler.CyclicLR(optimizer, base_lr=0.001,
max_lr=0.01, step_size_up=2000)

```

I hyper-tuned the learning rate to the maximum, momentum as well and the weigh_decay along with the base learning_rate and max learning rate. The values above gave me the best results , which for me was the validation accuracy. I tried using Adam instead of SGB as an optimizer, hyper-tuned it, but it was a few percenteges behing SGB at validation. Here is the exemple with Adam optimizer hyper-tuned:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.006,  
weight_decay=1e-5)  
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer,  
mode='min', factor=0.1, patience=5)
```

After I calculate the sum of losses over all batches in the current epoch by adding the multiplication of the scalar loss for the batch and the batch size which gives me the total loss for the batch, then get the predicted labels, compare to the actual correct labels and count the correct predictions. To the total number of samples, I sum the number of labels seen so far. At the end of the epoch I calculate the loss for that epoch by dividing the sum of losses for all batches in that epoch to the total number of labels seen so far. The accuracy is the correct number of predicted labels divided by the total number of labels seen so far. I append the epoch loss and accuracy in a list in order to plot them and view when the loss becomes linear and the accuracy stops growing in order to avoid overfit.

5. Testing

The same preprocessing is done as in the training, same for the data loader (same batch size, number of workers, but this time the data isn't shuffled) the model is changed to: model.eval() for evaluation and use with torch.no_grad() to disable gradient calculation since I don't need it besides in training so I lower my memory consumption and get the predicted labels for the images which are than stored in prediction.csv.

6. Validation

The same can be said about the validation as for the testing where I get the correctly predicted labels, divide that to the total number of labels and I get the accuracy.

7. Data Augmentation

In order to get a better validation accuracy, I decided to implement data augmentation. The original training set had 12500 images and I decided to augment the training set by another 12500 images. However, training the model for all 25000 images was very time consuming (over 3 hours) especially since I dump all the images at once in the training data loader (I had the memory and didn't see a reason to split the data). But, when augmenting finding the perfect values to augment the data in order to see improvements would take over 40 hours (since I had to change small values, train the model again, see results, fine tune again values) and that's where the K-NN model was extremely useful: it has a lower accuracy (by more than 20%), but it runs fast, in less than 2 minutes, so I added there as well the augmentation function to get and kept tweaking values, like rotation (now is 40) but I tried -40 , 35, 45, flip the image horizontally and basically mirror the image (left is right and right is left) and also flip it vertically (up is down and down is up), enhance the brightness by 18,5% tried with enhance(1.180 - to

1.187) , also tried reducing it (0.8 – 0.9) the sharpness is reduced (0.968) and the color too (0.8), this one made sense since the original images already have very vivid colors. Fine tuning all the parameters took more than 2 dozen tries, which would take an eternity for the CNN model, but only an hour or two max for the K-NN where I saw a 2 % increase in validation accuracy. I also tried adding gaussian noise, but it performed worse.

Here is how much it took (286 minutes) to train the model for 330 epochs in order to find a good one (186) to choose:

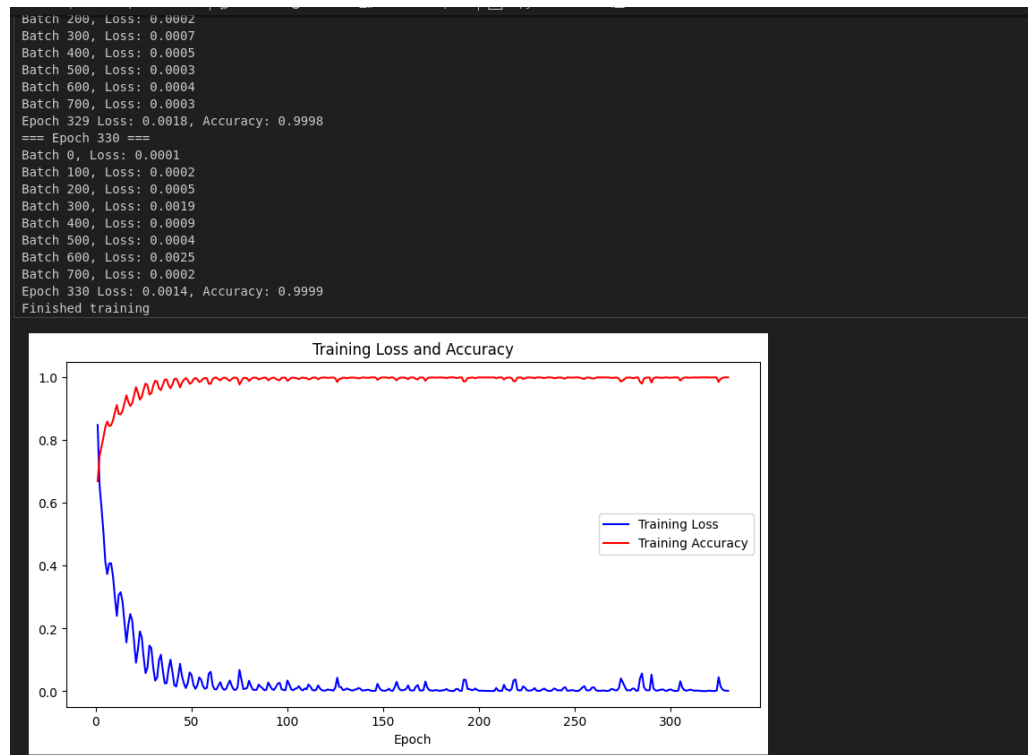
```

    if i % 100 == 0:
        print(f"Batch {i}, Loss: {loss.item():.4f}")

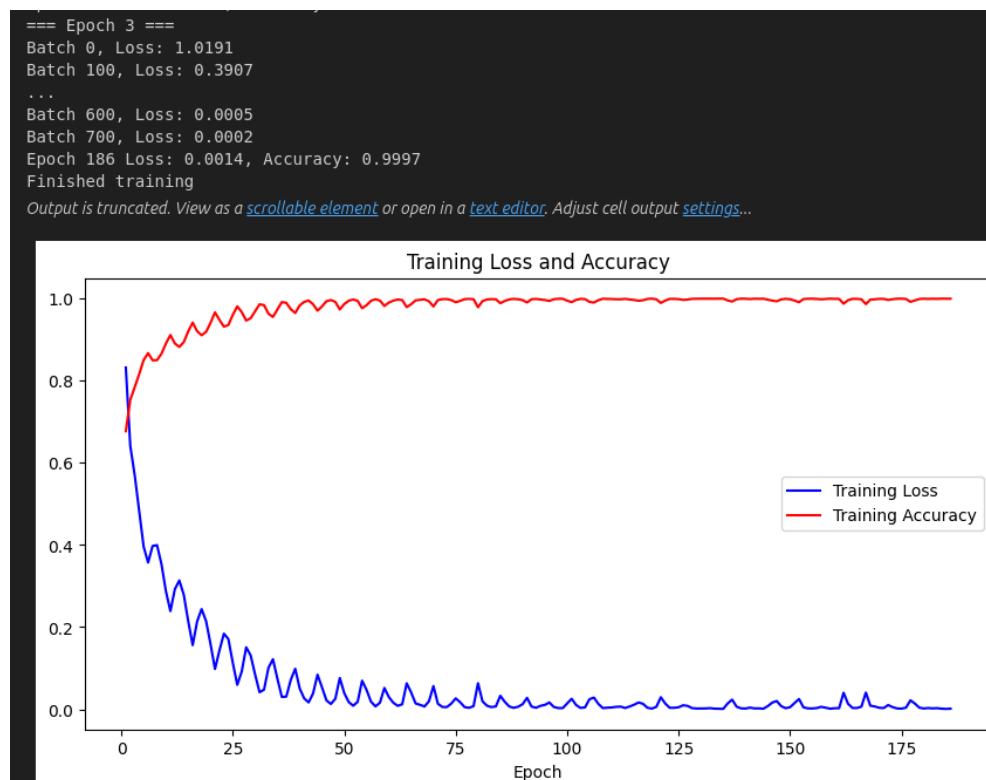
#####
epoch_loss = running_loss / total
epoch_acc = correct / total
train_losses.append(epoch_loss)
train_accuracies.append(epoch_acc)
# scheduler.step(epoch_loss)
print(f"Epoch {epoch+1} Loss: {epoch_loss:.4f}, Accuracy: {epoch_acc:.4f}")
# if epoch_loss < 0.001:
#     print(f"Stopping early at epoch {epoch+1} as loss is {epoch_loss:.4f}")
#     break
#####
print('Finished training')
epochs = range(1, len(train_losses) + 1)
plt.figure(figsize=(10, 5))
plt.plot(epochs, train_losses, 'b-', label='Training Loss')
plt.plot(epochs, train_accuracies, 'r-', label='Training Accuracy')
plt.xlabel('Epoch')
plt.title('Training Loss and Accuracy')
plt.legend()
plt.show()
✓ 208m 16.9s

Epoch 325 Loss: 0.0447, Accuracy: 0.9854
=== Epoch 326 ===
Batch 0, Loss: 0.0101
Batch 100, Loss: 0.0013
Batch 200, Loss: 0.0091
Batch 300, Loss: 0.0457
Batch 400, Loss: 0.0932
Batch 500, Loss: 0.0156
Batch 600, Loss: 0.0112
Batch 700, Loss: 0.0148
Epoch 326 Loss: 0.0159, Accuracy: 0.9949
=== Epoch 327 ===
Batch 0, Loss: 0.0001
Batch 100, Loss: 0.0011
Batch 200, Loss: 0.0005
Batch 300, Loss: 0.0066
Batch 400, Loss: 0.0012
Batch 500, Loss: 0.0018
```

And here is the plot with the training loss and accuracy



After choosing the epoch 186 based on the previous graph here is the final one which took 2 hours to run:

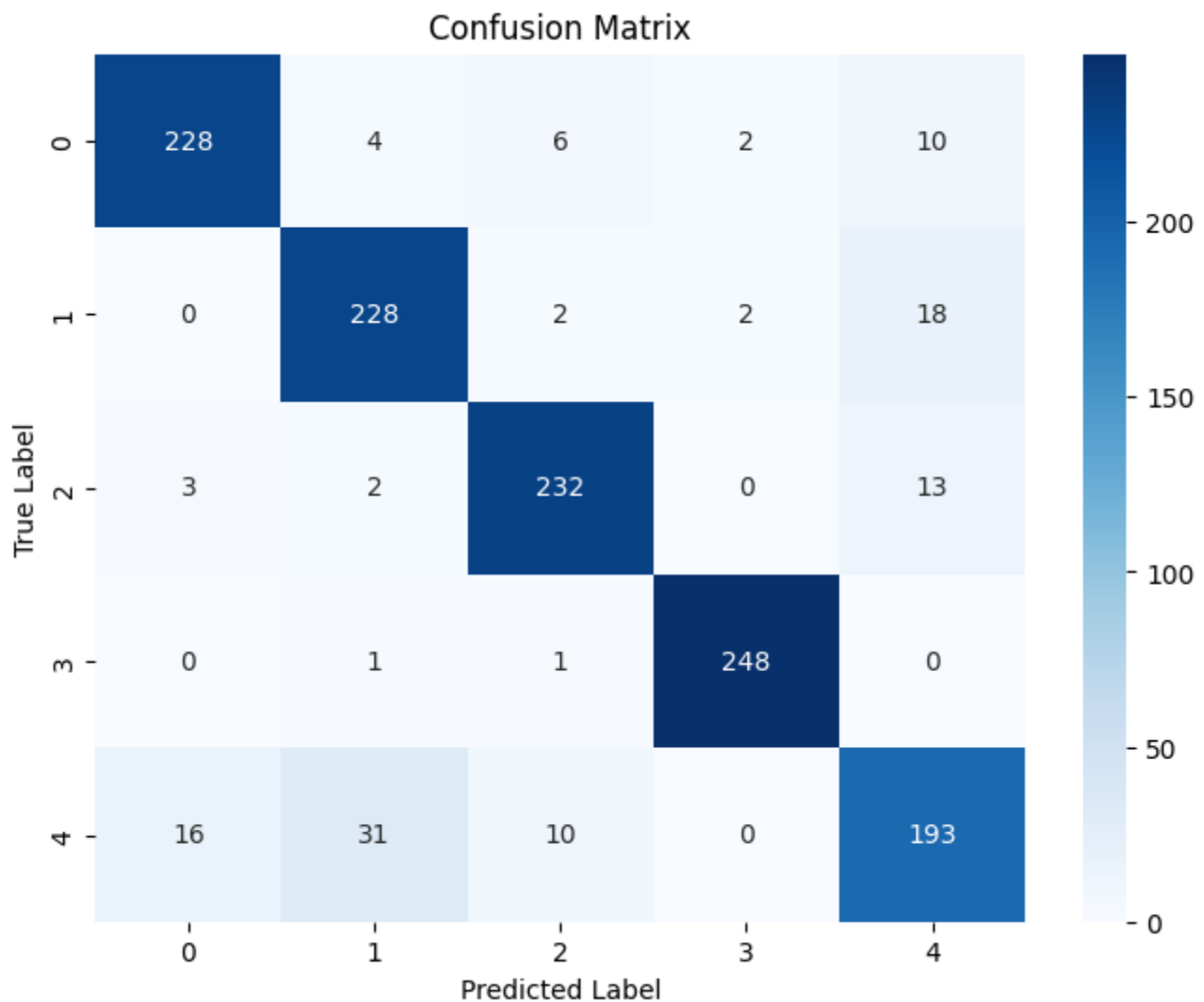


8. Confusion Matrix

I stored the validation predictions and the correct labels, then using `confusion_matrix` from `sklearn.metrics` I calculated the confusion matrix and printed it:

```
Confusion Matrix:  
[[228  4  6  2 10]  
 [  0 228  2  2 18]  
 [  3  2 232  0 13]  
 [  0  1  1 248  0]  
 [ 16 31 10  0 193]]
```

Used seaborn library to create a heatmap visualization so it looks better:



K-NN Model Description

1. Reasoning

I chose a K-NN as my second model since it's easy to implement and very fast compared to CNN, which gave me the opportunity to tinker with the augmentation data as I mentioned previously.

2. Data Reading and Processing

The same as previously mentioned in the CNN and the same augmentation, only here as a feature set I used histogram, which I first calculate, then normalize and flatten.

3. Training + Testing

As parameters after hyper-tuning I decided `n_neighbors = 5` (tried 7, 4, but worse validation accuracy), as metric I used `manhattan` (slower, but performed way better than `cosine`, by over 20%) and `weight = 'distance'` which makes the training set to have 100% accuracy, and over the validation set I saw 2% improvement. After that I get the training predictions in order to calculate the accuracy score (100% since I use `weight = 'distance'`). After I load the testing images get the predictions and store them in `knn_prediction.csv`

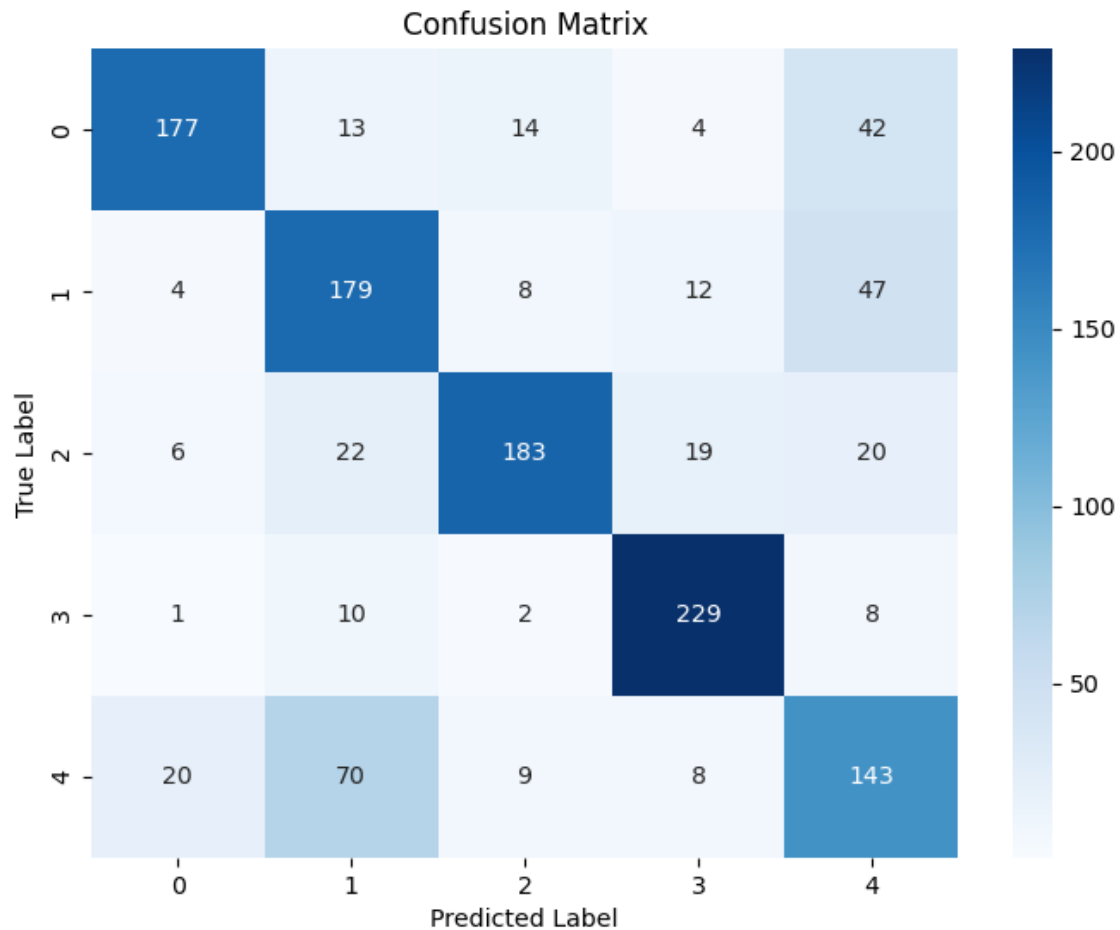
4. Validation

Same as testing, but I compute confusion matrix, print it and use `seaborn` module as in CNN to create a heatmap for better visualization.

Confusion Matrix Raw:

```
Finished generating augmented images
Training Accuracy: 100.00%
Validation Accuracy: 72.88%
Confusion Matrix:
[[177  13  14   4  42]
 [  4 179   8  12  47]
 [  6  22 183  19  20]
 [  1  10   2 229   8]
 [ 20  70   9   8 143]]
```

Confusion Matrix as heatmap:



Other side notes:

For CNN model I used Jupyter Notebook since it's takes too much time to train the model, and this helped me since I created a different cell for imports, paths from which I need to read training/ testing/ validation data, read training data, training, testing and for validation. However, I didn't use Jupyter Notebook for the K-NN model since the training took less than 2 minutes and I didn't consider it necessary.